

Compiler Design

A Complete Guide

Covering all six compiler phases, formal language theory,
data structures, optimization techniques, and modern architectures.

Contents

1	Introduction	2
1.1	The Compiler Pipeline	2
2	Phase 1 — Lexical Analysis (Scanning)	2
2.1	Key Definitions	3
2.2	Example	3
2.3	Implementation	3
3	Phase 2 — Syntax Analysis (Parsing)	3
3.1	Context-Free Grammars and BNF	3
3.2	Parse Tree Example	4
3.3	Parsing Strategies	4
4	Phase 3 — Semantic Analysis	4
4.1	Key Tasks	4
4.2	Attribute Grammars	4
5	Phase 4 — Intermediate Code Generation	5
5.1	Three-Address Code	5
5.2	Other IR Forms	5
6	Phase 5 — Code Optimization	5
6.1	Optimization Techniques by Scope	5
6.1.1	Local (Basic Block)	5
6.1.2	Global (Across Basic Blocks)	6
6.1.3	Machine-Level	6
7	Phase 6 — Code Generation	6
7.1	Key Tasks	6
8	The Symbol Table	6
8.1	What It Stores	6
8.2	Implementation	7
9	Formal Language Theory	7
9.1	The Chomsky Hierarchy	7
10	Error Handling	7
10.1	Parser Error Recovery Strategies	8
10.2	Error Categories by Phase	8
11	Key Data Structures	8
12	Important Algorithms	9
13	Modern Compiler Architectures	9
14	Learning Roadmap	9

Introduction

A **compiler** is a program that translates source code written in a high-level language into machine code (or another target language) that a computer can execute. It is one of the most elegant pieces of systems software ever built—a program that *understands* programs.

The compiler is classically divided into two halves:

- **Front end** (Phases 1–3): language-dependent; concerns itself with recognising valid programs.
- **Back end** (Phases 4–6): machine-dependent; concerns itself with producing efficient target code.

This separation is why compilers like LLVM can support dozens of languages *and* dozens of CPU architectures by sharing the middle representation.

The Compiler Pipeline

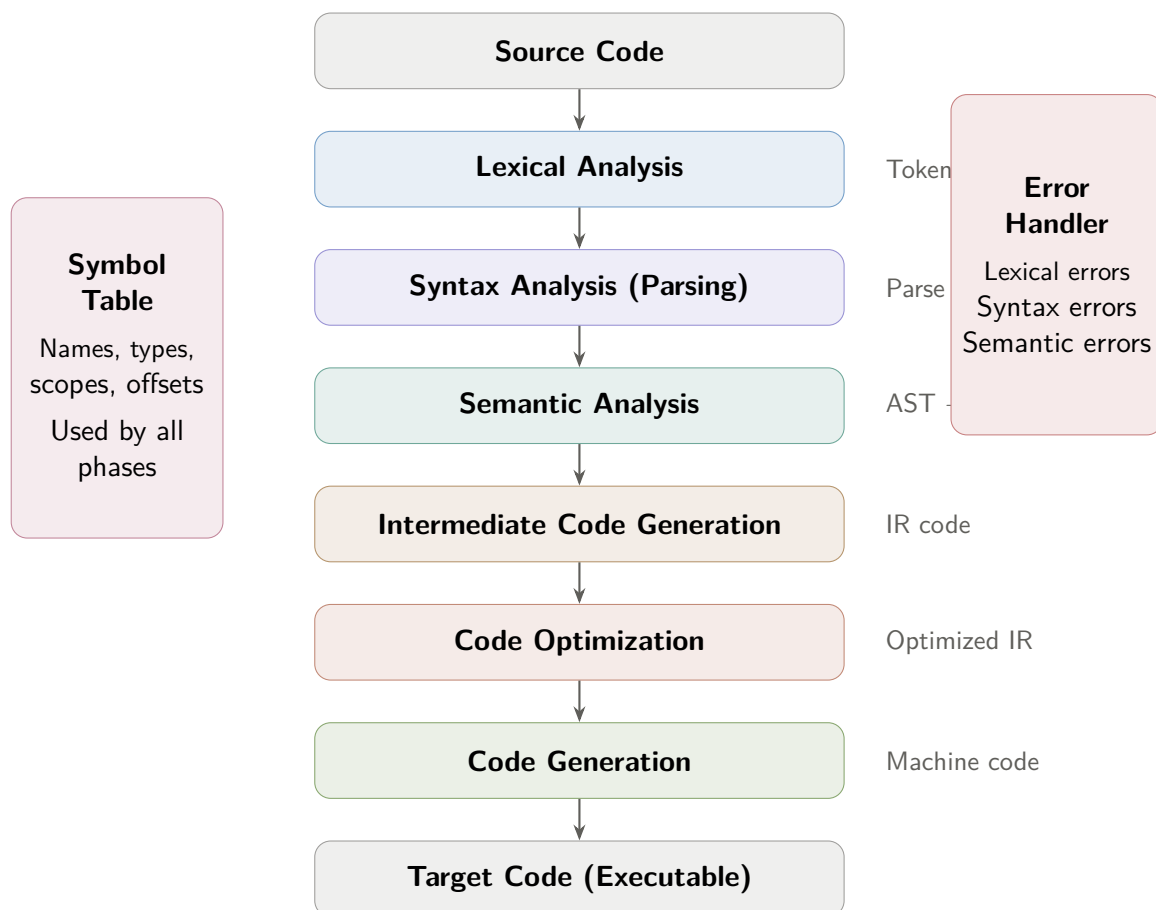


Figure 1: The six-phase compiler pipeline with cross-cutting components.

Phase 1 — Lexical Analysis (Scanning)

The **lexer** (scanner) reads raw characters and groups them into **tokens**—the smallest meaningful units of a language.

Key Definitions

Lexeme

The actual character string (e.g. "a", "5", "while").

Token

A category paired with an optional value, e.g. $\langle ID, "a" \rangle$, $\langle NUM, 5 \rangle$.

Pattern

The rule describing what lexemes match a token—usually a *regular expression*.

Finite Automaton

The mathematical model behind the scanner; regular expressions are converted to NFAs then DFAs for efficient scanning.

Example

Input	int x = a + 5;
Output tokens	INT, ID(x), ASSIGN, ID(a), PLUS, NUM(5), SEMICOLON

Common token categories: keywords (if, while), identifiers, literals (integers, floats, strings), operators, and punctuation. Whitespace is usually discarded.

Implementation

Regular expressions are converted to NFAs via **Thompson's Construction**, then to DFAs via the **Subset Construction**, and finally minimised. Tools such as `lex` and `flex` automate this process.

Phase 2 — Syntax Analysis (Parsing)

The **parser** takes the token stream and checks that it conforms to the **grammar** of the language, building a **parse tree** (concrete syntax tree).

Context-Free Grammars and BNF

Grammar is expressed in **Backus-Naur Form (BNF)**:

```
stmt  -> if ( expr ) stmt
        | while ( expr ) stmt
        | id = expr ;
expr  -> expr + term | term
term  -> id | num | ( expr )
```

Parse Tree Example

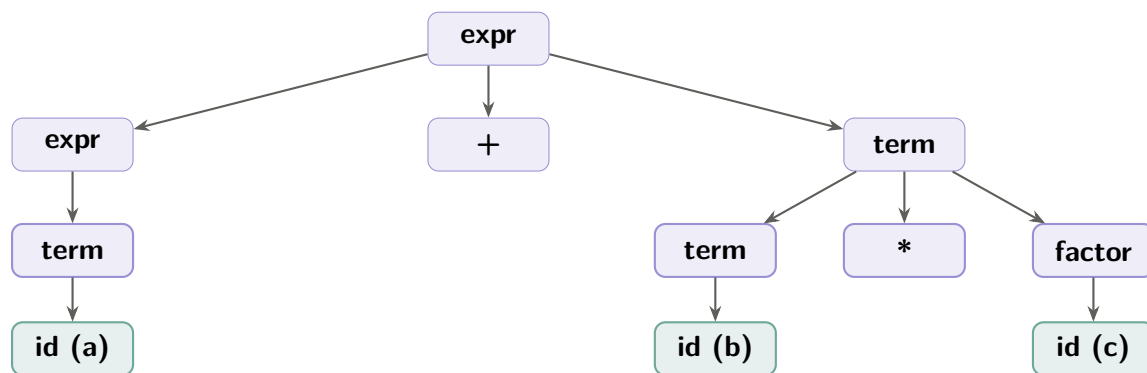


Figure 2: Parse tree for $a + b \times c$. The $*$ groups at a lower level, encoding higher precedence.

Parsing Strategies

Top-down (LL)

Starts from the start symbol and derives the input. LL(1) parsers use one lookahead token. Recursive descent parsers are hand-written LL parsers.

Bottom-up (LR)

Starts from the tokens and reduces toward the start symbol. Variants: LR(0), SLR(1), LALR(1), LR(1). Tools: yacc, bison.

Phase 3 — Semantic Analysis

Syntax alone does not catch every error. The semantic phase attaches *meaning* to the parse tree and produces an **Abstract Syntax Tree (AST)**—a parse tree stripped of redundant syntax nodes.

Key Tasks

- **Type checking** — Is "hello" + 5 legal in this language?
- **Scope resolution** — Is every variable declared before use?
- **Name binding** — Every use of x is linked to its declaration.
- **Type coercion** — Can `int` be implicitly cast to `float`?
- **Function signature checking** — Correct number and types of arguments?

Attribute Grammars

Context-Free Grammars handle syntax, but semantic rules need **Attribute Grammars**—annotations on CFG productions that carry type information *up* the tree (synthesized attributes) and *down* the tree (inherited attributes).

Phase 4 — Intermediate Code Generation

The front end produces an **Intermediate Representation (IR)**—a clean, machine-independent form. This portability is what allows one compiler infrastructure to target many architectures.

Three-Address Code

The most common IR form. Every instruction has at most one operator and three operands:

```
t1 = b * c
t2 = a + t1
x  = t2
```

Other IR Forms

SSA (Static Single Assignment)

Every variable is assigned exactly once. Used by modern compilers: LLVM IR, GCC GIMPLE.

Quadruples

Explicit (op, arg1, arg2, result) tuples stored in a table.

DAG (Directed Acyclic Graph)

For expression optimization within a basic block; automatically identifies common subexpressions.

Phase 5 — Code Optimization

The most intellectually rich phase. The goal is to transform the IR to run faster or use less memory without changing observable behaviour.

Optimization Techniques by Scope

6.1.1 Local (Basic Block)

Constant folding

Expressions such as $2*3+1$ are evaluated at compile time and replaced with the literal 7.

Common subexpression elimination (CSE)

If $a*b$ is computed twice in the same block, compute it once, store in a temp, and reuse.

Copy propagation

After $x = y$, replace uses of x with y ; enables further dead-code elimination.

Strength reduction

Replace expensive operations with cheaper ones: $x*2 \rightarrow x+x$, $x^2 \rightarrow x*x$.

Dead code elimination

Remove assignments whose results are never used.

6.1.2 Global (Across Basic Blocks)

Loop invariant code motion

Hoist computations that do not change inside a loop to before the loop.

Induction variable elimination

Replace $i*4$ in a loop body with a simpler variable updated by $+=4$.

Loop unrolling

Expand loop bodies to reduce branch overhead.

Inlining

Expand function bodies at call sites to eliminate call/return overhead.

6.1.3 Machine-Level

Register allocation

Minimise load/store instructions by maximising register reuse (see Section 7).

Peephole optimization

Scan small windows of generated instructions and replace with shorter or faster sequences.

Modern compilers (GCC, Clang) run dozens of passes at levels -O1, -O2, -O3.

Phase 6 — Code Generation

The final phase maps optimized IR to actual machine instructions for the target architecture.

Key Tasks

Instruction selection

Choosing which machine instructions implement each IR operation. One IR operation may map to several machine instructions (or vice versa).

Instruction scheduling

Reordering instructions to avoid pipeline stalls and exploit instruction-level parallelism.

Register allocation

An NP-complete problem in general, solved in practice with **graph coloring** (Chaitin's algorithm) or *linear scan*.

Memory layout

Stack frame layout, data alignment, and adherence to calling conventions.

The Symbol Table

The symbol table is the compiler's central data structure—a dictionary maintained throughout all phases.

What It Stores

For every declared name, the symbol table records:

- **Name** (identifier string)
- **Type** (int, float, pointer-to-struct, function signature, ...)
- **Scope level** (global, function-local, block-local)
- **Memory location** (offset from stack frame base, or absolute address)
- **Value** (for compile-time constants)

Implementation

Each scope is implemented as a **hash table** ($O(1)$ average lookup) with a **scope stack** pushed/popped as blocks are entered and exited. Nested scopes use *chained* symbol tables: the inner scope is searched first, then the outer.

Formal Language Theory

The Chomsky Hierarchy

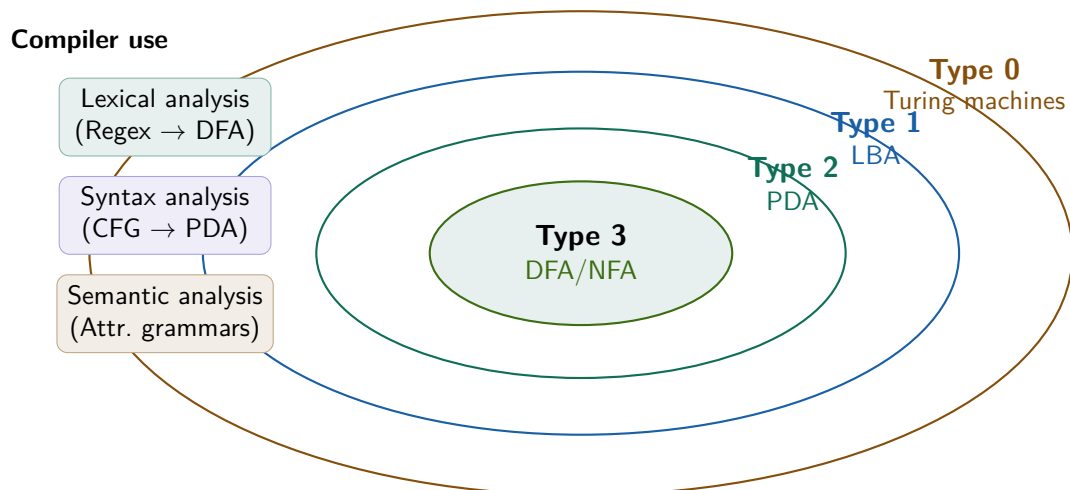


Figure 3: The Chomsky Hierarchy. Each level is a proper subset of the level above.

Type	Language class	Machine	Compiler use
3	Regular	DFA / NFA	Lexical analysis
2	Context-free	Pushdown automaton (PDA)	Parsing
1	Context-sensitive	Linear-bounded automaton	Semantic constraints
0	Recursively enum.	Turing machine	General computation

Table 1: Summary of the Chomsky Hierarchy and its relevance to compiler phases.

Error Handling

A production-quality compiler must not merely *detect* errors but *recover* from them and continue, so that as many errors as possible are reported in a single pass.

Parser Error Recovery Strategies

Panic mode

Discard tokens until a *synchronising token* (such as `;` or `}`) is found. Simple but loses context.

Phrase-level recovery

Locally correct the error (e.g. insert a missing semicolon), then continue. Risky if the wrong assumption is made.

Error productions

Add explicit grammar rules for common mistakes (e.g. $\text{expr} \rightarrow \text{expr} + + \text{expr}$) that produce a helpful diagnostic.

Error Categories by Phase

Phase	Example errors
Lexical	Unterminated string literal; invalid character @
Syntax	Missing <code>;</code> ; mismatched <code>{}</code>
Semantic	Type mismatch; undeclared variable; wrong argument count
Logical	<i>Not detected by the compiler</i> —only visible at runtime

Table 2: Error categories and their corresponding compiler phases.

Key Data Structures

Token stream

A list or iterator that flows from the lexer to the parser.

Parse tree / AST

A tree whose nodes represent grammar rules or language constructs.

Symbol table

A hash table per scope, linked in a scope chain.

Three-address code

A list of quadruples: `(op, arg1, arg2, result)`.

Control Flow Graph (CFG)

Nodes are basic blocks; edges are jumps. Used for global optimization and data-flow analysis.

Interference graph

For register allocation. Two variables share an edge if their lifetimes overlap; graph colouring assigns registers.

Important Algorithms

Algorithm	Purpose
Thompson's construction	Regex $\rightarrow$ NFA
Subset construction	NFA $\rightarrow$ DFA
DFA minimization	Minimize DFA states
Recursive descent	Top-down LL parsing
LR(0) / SLR / LALR / LR(1)	Bottom-up parsing table construction
Reaching definitions	Data-flow analysis
Dominator tree construction	SSA construction
Graph colouring (Chaitin)	Register allocation
Sethi-Ullman	Optimal expression evaluation order

Table 3: Key algorithms in compiler design.

Modern Compiler Architectures

GCC Multiple front ends (C, C++, Fortran, ...) $\rightarrow$ GIMPLE/RTL IR $\rightarrow$ multiple back ends.

LLVM

Clean IR (LLVM IR in SSA form) $\rightarrow$ modular optimization passes $\rightarrow$ many back ends (x86, ARM, WebAssembly, ...). Clang is LLVM's C/C++ front end.

JVM / CLR

Compile to portable bytecode (JVM bytecode, CIL), then JIT-compile to native code at runtime. Combines ahead-of-time and just-in-time compilation.

Craneflirt

A fast WASM/JIT compiler written in Rust, used in Firefox and Wasmtime.

Learning Roadmap

1. **Theory** — automata, regular expressions, context-free grammars.
Reference: Sipser, *Introduction to the Theory of Computation*.
2. **Classic reference** — Aho, Lam, Sethi, Ullman, *Compilers: Principles, Techniques, and Tools* ("Dragon Book").
3. **Modern practical** — Cooper & Torczon, *Engineering a Compiler* (clearer treatment of optimization).
4. **Build one** — write a lexer by hand, then a recursive descent parser, then code generation targeting a simple stack VM or x86.
5. **Study LLVM** — read LLVM IR, write an LLVM pass, use `clang -emit-llvm` to inspect what your code compiles to.